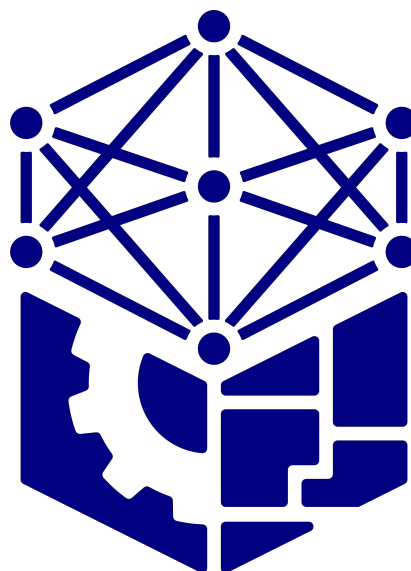


UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y
MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



Informe Final – Portal Digital TUPA UNSAAC

Asignatura:

Desarrollo de Software I

Docente:

Mg. Luis Alvaro Monzón Condori

Estudiante:

Castro Pari, Rayneld Fidel
Estacio Medrano, Amilcar
Florez Vega, Yamir Wagner
Merma Ccarhuarupay, Adel Alejandro

Cusco - Perú
2026



Índice

1	Introducción	1
1.1	Antecedentes	1
1.2	Planteamiento del Problema	1
1.3	Justificación	1
1.4	Alcance	1
1.5	Objetivos	2
1.5.1	Objetivo General	2
1.5.2	Objetivos Específicos	2
2	Marco Teórico	3
2.1	Arquitectura Cliente-Servidor y API REST	3
2.2	Node.js y Express	3
2.3	PostgreSQL	3
2.4	React y Vite	3
2.5	Autenticación basada en JSON Web Tokens (JWT)	4
2.6	Hash de Contraseñas	4
2.7	Metodología de Desarrollo	4
2.8	Multer y gestión de archivos	5
3	Desarrollo	5
3.1	Descripción General del Sistema	5
3.1.1	Usuarios y Roles	5
3.2	Requerimientos	6
3.2.1	Requerimientos Funcionales	6
3.2.2	Requerimientos No Funcionales	7
3.3	Arquitectura del Sistema	7
3.3.1	Arquitectura General	7
3.3.2	Arquitectura del Backend	7



3.3.3	Arquitectura del Frontend	8
3.4	Modelo de Base de Datos	9
3.4.1	Diccionario de Datos	9
3.4.2	Relaciones Principales	9
3.4.3	Restricciones de Integridad	11
3.5	Diseño de la API REST	11
3.5.1	Ciclo de Vida de una Solicitud	11
3.6	Diseño de Interfaces	12
3.6.1	Portal Público	13
3.6.2	Portal del Estudiante	13
3.6.3	Asistente de Solicitud	13
3.6.4	Portal Administrativo	20
3.6.5	Componente Pendiente	20
3.7	Seguridad	27
3.8	Despliegue	28
3.9	Pruebas	28
3.10	Historial de Commits	30
4	Conclusiones	30
4.1	Recomendaciones	31
	Referencias Bibliográficas	32

Introducción

Antecedentes

El Texto Único de Procedimientos Administrativos (TUPA) constituye el marco normativo que concentra los procedimientos administrativos de la UNSAAC. Dentro de este marco, los estudiantes y egresados deben gestionar trámites como constancias, convalidaciones, grados y títulos, entre otros, siguiendo requisitos, costos y plazos definidos por la institución. En la práctica, la solicitud y la revisión de estos procesos están asociadas a flujos administrativos documento-centricos, donde la evidencia de pago, los requisitos y la observación del expediente se registran y validan de manera manual y distribuida entre dependencias universitarias.

Planteamiento del Problema

La gestión del TUPA en la universidad presenta una brecha entre el marco normativo institucional y la experiencia de atención digital. A partir de la documentación del proyecto y de la estructura del repositorio, el problema central se observa en la falta de trazabilidad, la dependencia de validaciones manuales y la dispersión de la información sobre requisitos, estado y observaciones de cada trámite. Esto obliga a estudiantes y personal administrativo a coordinar procesos de forma menos eficiente, con mayor riesgo mínimo de errores de seguimiento y de demora en la atención.

Justificación

El desarrollo de una plataforma digital para la gestión del TUPA permite centralizar el catálogo de procedimientos, automatizar el seguimiento del estado de cada solicitud y reducir la dependencia de documentación física, beneficiando tanto a los estudiantes solicitantes como al personal administrativo encargado de la validación.

Alcance

El alcance del presente proyecto corresponde a una **demostración funcional (demo)** del sistema, orientada a validar el flujo completo de un trámite: desde la consulta

del catálogo público, la creación y llenado de una solicitud mediante un asistente guiado, la carga de documentos y el pago, hasta la revisión y decisión por parte del personal administrativo. En la versión implementada del repositorio, el flujo de pago se modela como la carga de un comprobante de pago en el backend, pero no existe integración con un proveedor de pagos real ni con la autenticación institucional de la UNSAAC. Asimismo, la ruta `/api/auth/google` queda como un stub con respuesta 501, por lo que la autenticación con Google está fuera del alcance de la demo entregada.

Objetivos

Objetivo General

Desarrollar una plataforma web para la gestión digital de procedimientos administrativos del TUPA de la UNSAAC, que permita a los estudiantes iniciar, dar seguimiento y completar sus trámites en línea, y al personal administrativo validarlos de forma centralizada.

Objetivos Específicos

1. Diseñar un modelo de base de datos relacional que represente los procedimientos del TUPA, sus requisitos, las solicitudes de los estudiantes y su historial de seguimiento.
2. Implementar una API REST en Node.js/Express que exponga las operaciones necesarias para el catálogo de trámites, la gestión de solicitudes, la carga de documentos y la administración de usuarios.
3. Implementar una interfaz web en React que permita a los estudiantes completar un asistente de trámite paso a paso y a los administradores gestionar la cola de solicitudes pendientes.
4. Incorporar mecanismos de autenticación y autorización basados en JWT que diferencien el acceso entre el rol de estudiante y el rol administrativo.

Marco Teórico

Arquitectura Cliente-Servidor y API REST

El sistema sigue una arquitectura cliente-servidor desacoplada, en la que el cliente (frontend) y el servidor (backend) se comunican exclusivamente a través de una interfaz de programación de aplicaciones (API) que expone recursos mediante el protocolo HTTP siguiendo los principios REST (Representational State Transfer): uso de verbos HTTP (GET, POST, PUT, PATCH, DELETE) sobre recursos identificados por rutas, intercambio de datos en formato JSON y ausencia de estado en el servidor entre peticiones, delegando la sesión del usuario al token enviado en cada solicitud.

Node.js y Express

Node.js es un entorno de ejecución de JavaScript del lado del servidor basado en el motor V8, orientado a operaciones de entrada/salida no bloqueantes. Express es un framework minimalista para Node.js que provee un sistema de enrutamiento y de middlewares para construir APIs HTTP (OpenJS Foundation, 2026). En el presente proyecto, Express se utiliza para definir los módulos de rutas, controladores y middlewares que conforman la API REST del backend.

PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales de código abierto que implementa el estándar SQL con soporte para transacciones ACID, llaves foráneas, restricciones de verificación (CHECK) e índices (The PostgreSQL Global Development Group, 2026). El modelo de datos del sistema se apoya en estas características para garantizar la integridad referencial entre solicitudes, usuarios, trámites y documentos.

React y Vite

React es una biblioteca de JavaScript para la construcción de interfaces de usuario basada en componentes reutilizables y un modelo declarativo de renderizado (Meta Plat-

forms, Inc., 2026). Vite es la herramienta de compilación y servidor de desarrollo utilizada para el proyecto frontend, que ofrece recarga en caliente y un empaquetado optimizado para producción.

Autenticación basada en JSON Web Tokens (JWT)

JWT es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información entre dos partes como un objeto JSON firmado digitalmente (Jones et al., 2015). En el sistema, el token JWT se emite tras un inicio de sesión exitoso y contiene el identificador del usuario y su rol, permitiendo validar cada solicitud subsecuente sin necesidad de mantener sesiones en el servidor.

Hash de Contraseñas

Para el almacenamiento seguro de credenciales, las contraseñas no se guardan en texto plano, sino como el resultado de una función de hash con salteo (*salt*) mediante el algoritmo `bcrypt`, diseñado específicamente para dificultar ataques de fuerza bruta mediante un factor de costo configurable.

Metodología de Desarrollo

El desarrollo del proyecto se organizó de forma iterativa e incremental, construyendo primero el modelo de datos, luego la capa de API y finalmente la integración con la interfaz de usuario. La evidencia verificable en el repositorio corresponde a una estructura modular de trabajo por carpetas (`backend/` y `frontend/`), con definición formal de rutas y servicios y con un flujo de validación local mediante ejecución del servidor y pruebas automatizadas. No se encuentra evidencia comprobable en el proyecto de un tablero Kanban o de una herramienta de gestión de sprints distinta a la organización académica y de entrega del curso, por lo que el enfoque se describe como una metodología de desarrollo ágil informal, apoyada en la implementación continua y en pruebas de integración HTTP.

Multer y gestión de archivos

La carga de documentos y comprobantes de pago se implementa en el backend mediante la librería `multer`, un middleware de Express para la recepción de archivos `multipart/form-data`

`parentitemulterdocs`. En el repositorio, esta capa se configura con almacenamiento en disco y validación de tipo de archivo para aceptar únicamente PDF, JPG y PNG, con límite de tamaño de 5 MB por carga.

Desarrollo

Descripción General del Sistema

El sistema, denominado Portal Digital TUPA - UNSAAC, está compuesto por tres capas: una aplicación cliente construida en React que consume una API REST desarrollada en Node.js/Express, la cual a su vez persiste la información en una base de datos relacional PostgreSQL. El sistema contempla dos tipos de usuario con necesidades y permisos distintos.

Usuarios y Roles

El *payload* del token JWT distingue únicamente dos roles, USER y ADMIN, que son los que gobiernan el control de acceso de la API. El campo `rol_admin` de la tabla `usuario_admin` (ADMIN o SUPER_ADMIN) se transporta en el token como `subRole` pero, en la versión entregada, no diferencia permisos.

- **Estudiante** — rol USER, tabla `usuario_general`: puede consultar el catálogo público de trámites, iniciar una solicitud mediante un asistente guiado de seis pasos, cargar los documentos requeridos, registrar su comprobante de pago, dar seguimiento a sus expedientes y recibir notificaciones sobre el estado de estos.
- **Administrador** — rol ADMIN, tabla `usuario_admin`: puede revisar la cola de solicitudes pendientes, validar o rechazar los documentos cargados, registrar observaciones y decisiones sobre cada expediente, y gestionar el catálogo de trámites y los usuarios del sistema.

Requerimientos

Requerimientos Funcionales

Cuadro 1: Requerimientos funcionales del sistema y su estado de implementación.

Código	Descripción	Estado
RF-01	Consulta pública del catálogo de trámites, con filtros por búsqueda, categoría, costo y plazos.	Implementado
RF-02	Inicio de sesión de estudiantes y administradores mediante credenciales del sistema.	Implementado
RF-03	Registro de cuenta con correo institucional y activación por enlace enviado por correo.	Implementado
RF-04	Recuperación de contraseña mediante código de seis dígitos enviado por correo.	Implementado
RF-05	Creación de una solicitud en borrador y avance del asistente por los pasos 1 a 6.	Implementado
RF-06	Carga de comprobante de pago y documentos requeridos por requisito.	Implementado
RF-07	Generación automática de número de expediente al confirmar el envío.	Implementado
RF-08	Rastreo público por número de expediente, con datos del solicitante anonimizados.	Implementado
RF-09	Notificaciones de estado y observaciones para el usuario.	Implementado
RF-10	Cola administrativa de solicitudes pendientes con revisión por expediente.	Implementado
RF-11	Decisión administrativa sobre validación de documentos y estado de la solicitud.	Implementado
RF-12	Gestión del catálogo y activación/desactivación de usuarios y trámites.	Implementado
RF-13	Integración con pasarela de pagos y autenticación con Google.	No implementado

La tabla anterior se valida directamente con las rutas expuestas por el backend (/api/auth, /api/procedures, /api/requests, /api/admin) y con los componentes de navegación definidos en frontend/src/App.jsx. El requerimiento RF-13 queda fuera del alcance de la versión entregada: el endpoint POST /api/auth/google responde 501 Not Implemented de forma explícita, y el pago se registra mediante la carga manual de un comprobante en lugar de una pasarela.

Cuadro 2: Requerimientos no funcionales del sistema.

Código	Descripción
RNF-01	Las contraseñas de los usuarios deben almacenarse cifradas mediante una función de hash (bcrypt, 10 rondas de sal), nunca en texto plano.
RNF-02	El acceso a los recursos protegidos de la API debe requerir un token JWT válido en la cabecera <i>Authorization</i> .
RNF-03	Las rutas bajo <i>/api/admin</i> deben restringirse a portadores de un token con rol ADMIN.
RNF-04	Todas las consultas a la base de datos deben usar sentencias parametrizadas para prevenir inyección SQL.
RNF-05	Los endpoints sensibles de autenticación deben limitar la cantidad de intentos por dirección IP dentro de una ventana de tiempo.
RNF-06	Los archivos cargados deben restringirse por tipo (PDF, JPG, JPEG, PNG) y por tamaño máximo (5 MB).
RNF-07	Los mensajes de error de autenticación no deben permitir deducir qué correos están registrados en el sistema.
RNF-08	La interfaz debe ser consumible desde un navegador web moderno, comunicándose con el backend mediante peticiones HTTP habilitadas por CORS.

Requerimientos No Funcionales

Arquitectura del Sistema

Arquitectura General

La Figura 1 presenta la arquitectura general del sistema, organizada en tres capas desacopladas que se comunican mediante peticiones HTTP/JSON.

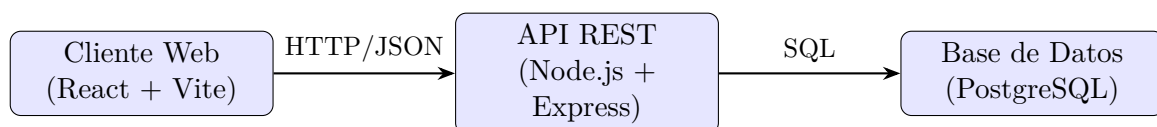


Figura 1: Arquitectura general del sistema en tres capas.

Arquitectura del Backend

La organización real del backend en el repositorio es consistente con un patrón Rutas–Controladores–Servicios, pero además incorpora utilidades y miembros adicionales de infraestructura:

- `routes/`: define los siete prefijos de la API (`/api/auth`, `/api/users`, `/api/procedures`,

`/api/requests`, `/api/documents`, `/api/notifications` y `/api/admin`) y enlaza cada ruta con su controlador.

- `controllers/`: reciben la solicitud HTTP, delegan la lógica a los servicios y responden con JSON. Todos los manejadores se envuelven en `asyncHandler`, de modo que cualquier promesa rechazada llega al middleware de errores en lugar de quedar sin capturar.
- `services/`: contienen la lógica de negocio y las consultas parametrizadas sobre PostgreSQL, incluidas la creación de solicitudes, la generación del expediente, la validación documental y la gestión de usuarios y trámites. Las operaciones que tocan varias tablas (registro, envío de solicitud, decisión administrativa) se ejecutan dentro de una transacción con `BEGIN/COMMIT` y bloqueo `SELECT ...FOR UPDATE` sobre la fila afectada.
- `middleware/`: implementa la autenticación JWT (`auth.middleware`), la autorización por rol, el limitador de intentos por IP (`rateLimit.middleware`), la carga de archivos mediante `multer` (`upload.middleware`) y el manejador centralizado de errores (`error.middleware`), que traduce los códigos de restricción de PostgreSQL a códigos HTTP de cliente.
- `db/`: centraliza el *pool* de conexiones a PostgreSQL.
- `utils/`: contiene la generación del número de expediente (`generateExpediente`), la firma y verificación de tokens JWT (`jwt.util`) y las validaciones de entrada compartidas (`validate`), que normalizan identificadores y paginación.

Arquitectura del Frontend

La organización real del frontend presenta una división funcional por tipo de vista, pero con una estructura más detallada que la descrita de forma general:

- `pages/public`: páginas de acceso público como landing, catálogo, búsqueda y rastreo de expedientes.
- `pages/auth`: flujo de autenticación (inicio de sesión, registro, activación de cuenta y recuperación de contraseña).

- `pages/student`: panel del estudiante, perfil, notificaciones, trámites propios y detalle de solicitudes.
- `pages/student/wizard`: asistente del flujo de solicitud en seis pasos.
- `pages/admin`: panel administrativo, cola de pendientes, validación documental, gestión de procedimientos, usuarios y reportes.
- `components/layout`: envolturas de página y el guardián de rutas `RequireAuth`, que redirige al inicio de sesión cuando no hay sesión activa y al panel correspondiente cuando el rol no coincide.
- `context/`: estado global de sesión (`AuthContext`) y del asistente de trámites (`WizardContext`), este último persistido en `localStorage` para que el avance no se pierda al recargar la página.
- `lib/api.js`: cliente HTTP único que adjunta el token de acceso, renueva automáticamente la sesión cuando expira y encola las peticiones que fallan mientras dura la renovación.

Modelo de Base de Datos

Diccionario de Datos

El modelo relacional está compuesto por catorce entidades: once del dominio del negocio y tres dedicadas a la gestión de tokens de autenticación. La Tabla 3 resume su propósito dentro del sistema.

Relaciones Principales

Un `usuario_general` puede tener múltiples `solicitud` (una por trámite iniciado); cada `solicitud` referencia un `tramite` del catálogo. Cada `tramite` define uno o más `requisito`, y cada documento cargado por el estudiante se asocia opcionalmente a un `requisito` específico dentro de una `solicitud`; los comprobantes de pago se registran como documentos sin requisito asociado. Las tablas `observacion` y `seguimiento` registran, respectivamente, las incidencias y el historial de cambios de estado de una `solicitud`, ambas vinculadas al `usuario_admin` que las generó.

Cuadro 3: Diccionario de datos del modelo relacional.

Entidad	Descripción
especialidad	Catálogo de especialidades/facultades a las que puede pertenecer un estudiante.
usuario_general	Estudiantes que solicitan trámites; incluye datos institucionales, de contacto y el avatar almacenado como BYTEA.
usuario_admin	Personal administrativo del sistema, con rol ADMIN o SUPER_ADMIN.
refresh_token	Tokens de renovación de sesión emitidos, con marca de revocación y vencimiento.
activation_token	Enlaces de activación de cuenta, de un solo uso y con vigencia de 24 horas.
password_reset_token	Códigos de recuperación de contraseña, con contador de intentos y vigencia de 30 minutos.
categoria	Agrupación temática de los trámites del catálogo (p. ej. Certificados, Matrícula).
tramite	Catálogo oficial de procedimientos del TUPA: costo, plazo, base legal y vigencia.
requisito	Requisitos documentales obligatorios u opcionales asociados a cada trámite.
solicitud	Expediente creado por un estudiante para un trámite: estado, prioridad, número de expediente.
documento	Archivo digital cargado por el estudiante para cumplir un requisito específico; el contenido se guarda en la columna contenido de tipo BYTEA.
observacion	Observación registrada por un administrador sobre un documento o solicitud.
seguimiento	Bitácora histórica de cambios de estado de una solicitud.
notificacion	Alerta generada hacia un estudiante ante un evento relevante de su solicitud.

Restricciones de Integridad

El modelo aplica restricciones CHECK para limitar los valores válidos de los campos de estado, evitando así estados inconsistentes a nivel de base de datos:

- `solicitud.estado` $\in \{\text{BORRADOR, SOLICITADO, VERIFICANDO_PAGO, PAGADO, EN PROCESO, SUBSANACION, OBSERVADO, COMPLETADO, ANULADO, RECHAZADO}\}$.
- `solicitud.paso_actual` restringido al intervalo $[1, 6]$, de modo que el avance del asistente no puede salirse de rango.
- `documento.estado_validacion` $\in \{\text{PENDIENTE, APROBADO, RECHAZADO}\}$.
- `observacion.estado` $\in \{\text{PENDIENTE, SUBSANADO, CERRADO}\}$.
- `refresh_token.rol` y `password_reset_token.rol` $\in \{\text{USER, ADMIN}\}$.

Adicionalmente, `solicitud.numero_expediente` lleva una restricción UNIQUE, y el esquema define índices sobre las columnas de búsqueda frecuente (estado, fecha, expediente, usuario) y un índice parcial sobre las notificaciones no leídas. El script de creación completo se encuentra en `backend/sql/schema.sql`.

Diseño de la API REST

La API se organiza en siete módulos funcionales, cada uno bajo su propio prefijo de ruta, que suman 38 endpoints. La Tabla 4 resume los módulos implementados y su conjunto de operaciones definido en la capa de rutas del backend.

Las rutas públicas son el catálogo completo (`/api/procedures`), el rastreo por expediente (`GET /api/requests/track/:numero_expediente`), la descarga de avatares y todo el módulo de autenticación. El resto exige un token válido, y las ocho rutas de `/api/admin` exigen además rol ADMIN.

Ciclo de Vida de una Solicitud

El expediente avanza mediante un conjunto acotado de transiciones. Al crear el borrador la solicitud nace en BORRADOR; la carga del comprobante la mueve a VERIFICANDO_PAGO; el envío (`POST /api/requests/:id/submit`) verifica que todos los requisitos obligatorios tengan documento adjunto, genera el número de expediente con formato EXP-AAAA-NNNNNN

Cuadro 4: Endpoints de la API REST del backend.

Módulo	Ruta	Endpoints implementados
Autenticación	/api/auth	POST /login, POST /refresh, POST /logout, POST /register, POST /activate/:token, POST /resend-activation, POST /forgot-password, POST /resend-code, POST /verify-code, POST /reset-password, POST /google (stub, responde 501).
Perfil de usuario	/api/users	GET /profile, PUT /profile, PUT /profile/password, POST /profile/avatar, GET /avatar/:role/:id.
Catálogo de trámites	/api/procedures	GET /, GET /categories, GET /:cod_tramite.
Solicitudes	/api/requests	POST /, GET /, GET /:id, PATCH /:id/step, POST /:id/voucher, POST /:id/document/:id_requisito, POST /:id/submit, GET /track/:numero_expediente.
Documentos	/api/documents	GET /:id_documento/view, DELETE /:id_documento.
Notificaciones	/api/notifications	GET /, POST /read-all, PATCH /:id/read.
Administración	/api/admin	GET /stats, GET /requests, GET /requests/:id, POST /requests/:id/decision, POST /procedures, PATCH /procedures/:cod_tramite/toggle, GET /users, PATCH /users/:id/toggle.

y deja la solicitud en SOLICITADO. A partir de ahí, la decisión administrativa (POST /api/admin/requests/:id/decision) admite las acciones APROBAR, OBSERVAR, RECHAZAR y EN_PROCESO. La acción APROBAR sólo cierra el expediente como COMPLETADO si todos los requisitos obligatorios tienen su documento en estado APROBADO; en caso contrario lo deja en EN_PROCESO. Los estados COMPLETADO, RECHAZADO y ANULADO son terminales y no admiten nuevas decisiones. Cada transición inserta un registro en seguimiento y genera una notificación para el estudiante.

Diseño de Interfaces

El frontend implementado en React y Vite expone 33 rutas para los principales escenarios del sistema. Las capturas que siguen fueron obtenidas navegando la aplicación en ejecución mediante un navegador automatizado, con el backend conectado a la base de datos del proyecto; por tanto, los datos que muestran son los reales del sistema y no maquetas.

Portal Público

La navegación pública incluye la página de inicio, el catálogo de trámites, el registro, la recuperación de contraseña y la consulta de expedientes, a través de los componentes `LandingPage.jsx`, `Catalog.jsx`, `Registro.jsx`, `RecuperarPassword.jsx`, `TrackingSearch.jsx` y `TrackingResults.jsx`.



Figura 2: Página de inicio del portal público (ruta `/`).

Portal del Estudiante

El portal del estudiante incorpora el tablero principal, el perfil, el centro de notificaciones, el listado de trámites y solicitudes y el detalle de cada expediente. En la estructura del frontend, estos módulos se materializan en `Dashboard.jsx`, `MyProfile.jsx`, `Notifications.jsx`, `MyProcedures.jsx`, `MyRequests.jsx` y `RequestDetail.jsx`.

Asistente de Solicitud

El envío de una solicitud se realiza mediante un asistente de seis pasos (`wizard/Step1.jsx` a `Step6.jsx`), cuyo estado se comparte entre rutas a través de `WizardContext`: selección del trámite, revisión de requisitos, confirmación de pago, carga de documentos,

TUPA UNSAAC
Portal Administrativo Digital

Gestiona tus trámites universitarios en un solo lugar

Accede al catálogo completo de procedimientos, rastrea el estado de tus expedientes y recibe notificaciones en tiempo real.

- Trámites 100% Digitales**
Sin colas, sin papel, sin esperas
- Seguimiento en Tiempo Real**
Estado actualizado de cada expediente
- Acceso Seguro e Institucional**
Cifrado SSL y autenticación UNSAAC

103+ Procedimientos 24/7 Disponibilidad 15K+ Usuarios

TUPA Central
Gestión de Procedimientos Administrativos - UNSAAC

TIPO DE ACCESO

Estudiante
Pregrado / Posgrado

Administrativo
Personal UNSAAC

Correo institucional o CUI
ej. nombre.apellido@unsaac.edu.pe

Contraseña

¿Olvidaste tu contraseña?

☐ Mantener sesión iniciada

Iniciar Sesión

o continuar con

Google Workspace UNSAAC

¿No tienes cuenta? Solicitar acceso institucional

Acceso SSL Seguro Uso Institucional Datos Protegidos

Figura 3: Inicio de sesión con selector de tipo de usuario (ruta /login).

TUPA UNSAAC
Portal Administrativo Digital

Tus trámites universitarios, en un solo lugar

Presenta solicitudes, adjunta tus documentos y sigue el estado de cada expediente sin moverte de casa.

Crear cuenta

Regístrate con tu correo institucional para presentar trámites en línea

Nombres * Apellido paterno *

Apellido materno DNI *

Código universitario Teléfono

20201234 984000000

Correo institucional *

nombre.apellido@unsaac.edu.pe

Debe terminar en @unsaac.edu.pe

Contraseña *

Repetir contraseña *

Crear cuenta

¿Ya tienes cuenta? Inicia sesión

Figura 4: Registro de cuenta con validación de correo institucional (ruta /registro).

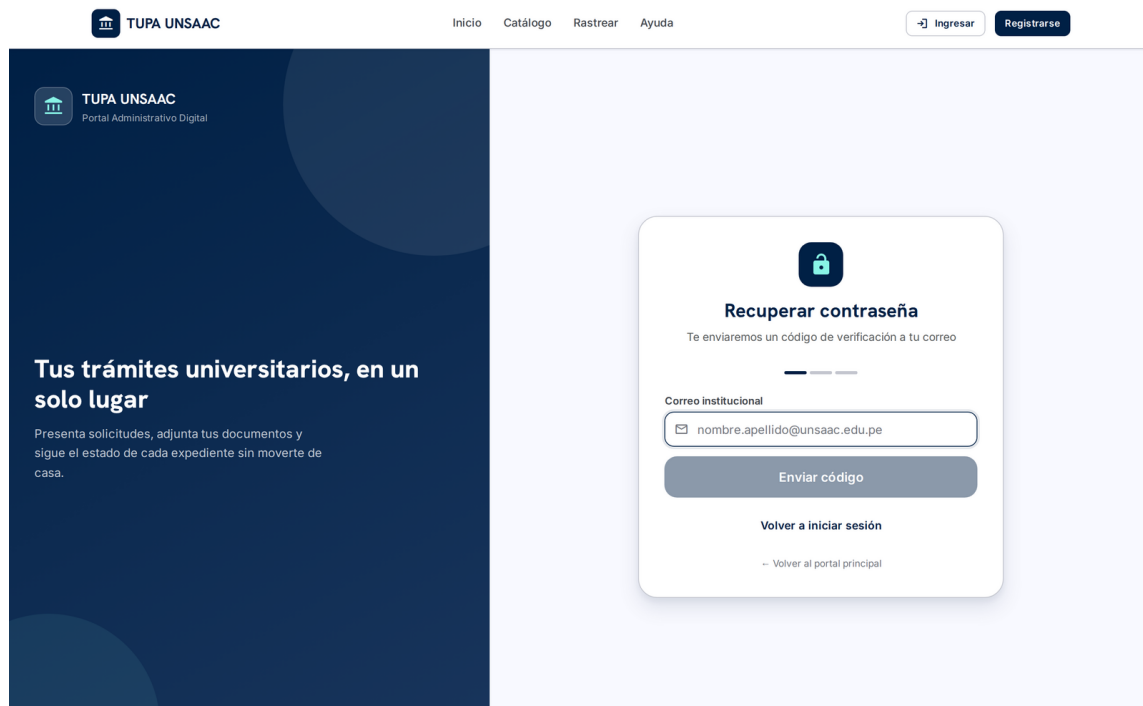


Figura 5: Recuperación de contraseña mediante código de verificación (ruta /recuperar-password).

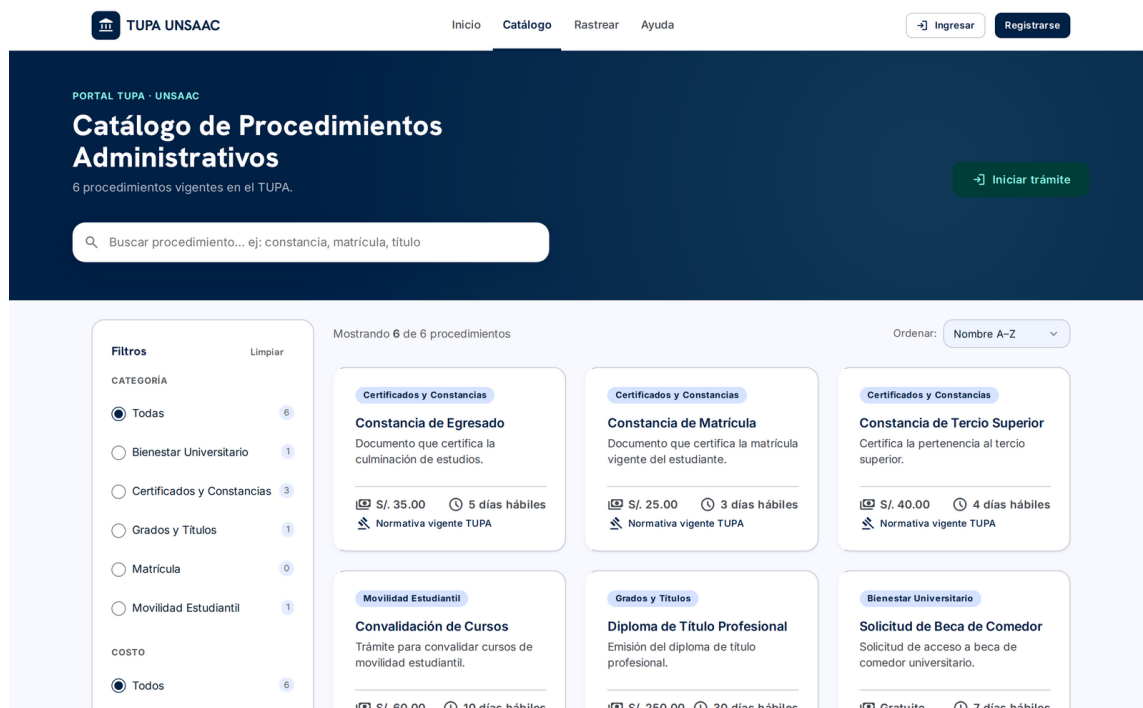


Figura 6: Catálogo público de trámites del TUPA (ruta /catalogo).

TUPA UNSAAC Inicio Catálogo Rastrear Ayuda Ingresar Registrarse

SEGUIMIENTO DE EXPEDIENTES

Rastrear mi Expediente

Ingresa el código de tu expediente para conocer el estado actual de tu trámite en tiempo real.

Ej. EXP-2026-000001

Formato: EXP-AAAA-NNNNNN · Ej. EXP-2026-000001

¿Cómo encontrar tu código de expediente?

Correo de confirmación
Al enviar tu solicitud recibiste un correo con el código de expediente.

Panel del estudiante
Inicia sesión y ve a "Mis Trámites" para ver todos tus expedientes.

Comprobante de pago
El código aparece en el comprobante de pago del trámite.

¿Ya tienes cuenta? Accede directamente a tus expedientes.

Figura 7: Consulta pública de expediente (ruta /seguimiento).

TUPA UNSAAC Inicio Catálogo Rastrear Ayuda Ingresar Registrarse

EXP-2026-000013

Expediente EXP-2026-000013 encontrado En vivo

Solicitud de Beca de Comedor Completado

EXP-2026-000013

Solicitante	Etapas actual	Fecha de ingreso	Plazo
Amilcar E.	Finalizado	04 ago. 2026	7 días hábiles

Historial público

- En proceso**
bien hecho mikin
04 ago. 2026 - 09:21 a. m.
- En proceso**
04 ago. 2026 - 09:22 a. m.
- Completado**
ahora si aprobado mikin
04 ago. 2026 - 09:23 a. m.

Figura 8: Resultado de la consulta pública, con el nombre del solicitante abreviado (ruta /seguimiento/resultados).

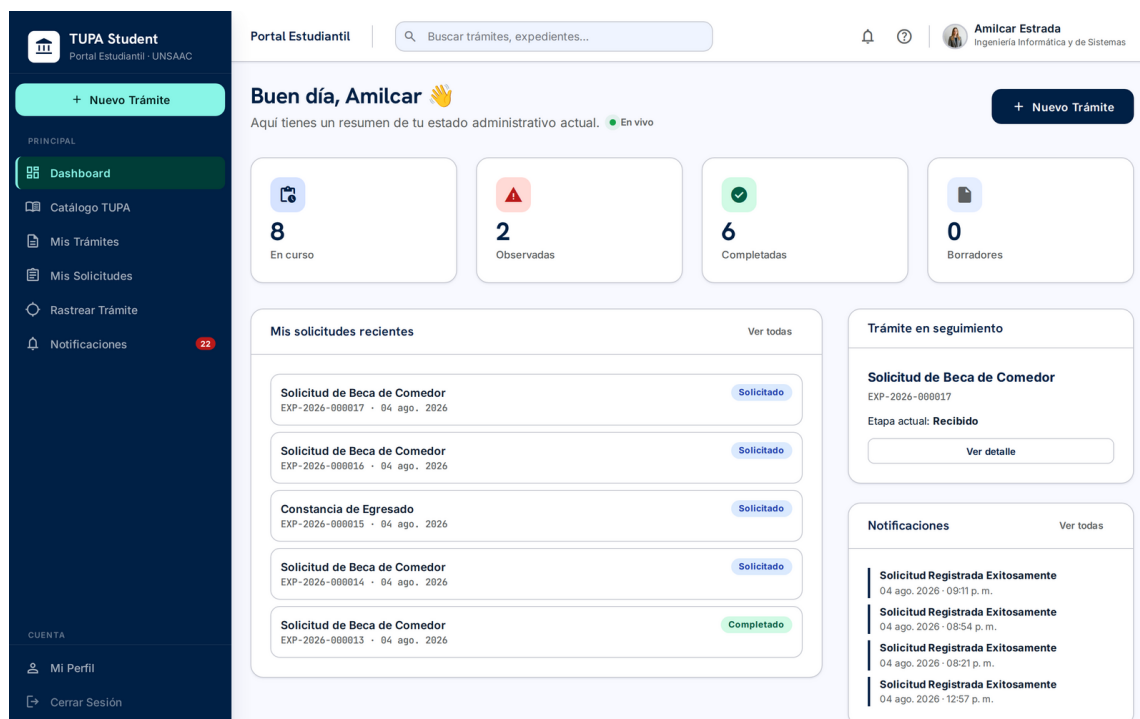


Figura 9: Tablero principal del estudiante (ruta /estudiante).

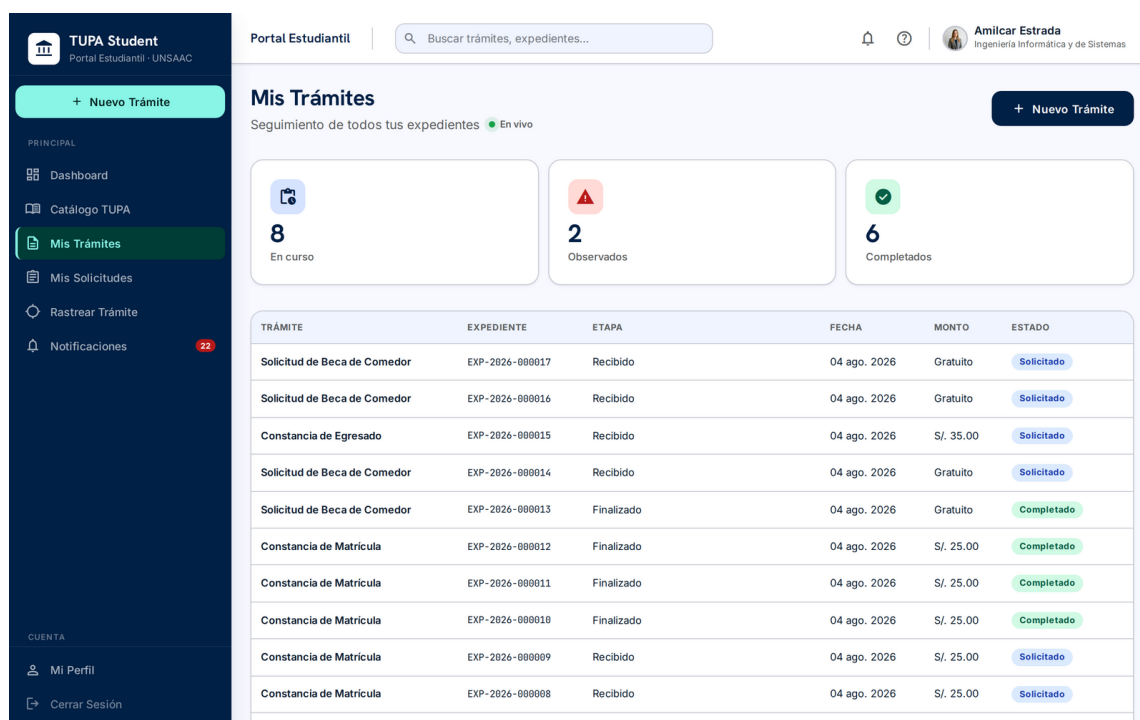


Figura 10: Catálogo de trámites disponibles dentro del portal del estudiante (ruta /estudiante/tramites).

The screenshot shows the 'Mis Solicitudes' (My Requests) page of the TUPA Student Portal. The left sidebar contains navigation links: Dashboard, Catálogo TUPA, Mis Trámites, Mis Solicitudes (active), Rastrear Trámite, and Notificaciones (22). The main content area displays a grid of request cards. Each card shows the request title, status (e.g., 'Solicitado', 'En subsanación', 'Observado'), EXP number, receipt date, and amount. A 'Ver detalle' button is present on each card. The top right shows the user's name 'Amilcar Estrada' and a search bar.

Request Title	Status	EXP	Received Date	Amount
Solicitud de Beca de Comedor	Solicitado	EXP-2026-000017	04 ago. 2026	Gratuito
Solicitud de Beca de Comedor	Solicitado	EXP-2026-000016	04 ago. 2026	Gratuito
Constancia de Egresado	Solicitado	EXP-2026-000015	04 ago. 2026	S/. 35.00
Solicitud de Beca de Comedor	Solicitado	EXP-2026-000014	04 ago. 2026	Gratuito
Constancia de Matrícula	Solicitado	EXP-2026-000009	04 ago. 2026	S/. 25.00
Constancia de Matrícula	Solicitado	EXP-2026-000008	04 ago. 2026	S/. 25.00
Constancia de Egresado	Observado	EXP-2026-000007	04 ago. 2026	Requiere Subsanación
Constancia de Egresado	En subsanación	EXP-2026-000004	04 ago. 2026	Observado

Figura 11: Listado de solicitudes del estudiante con su estado actual (ruta /estudiante/solicitudes).

The screenshot shows the details of a specific request, 'Constancia de Egresado' (EXP-2026-000004), which is in 'En subsanación' (Under Review) status. A yellow warning banner indicates that the request has observations for subsanation: 'El récord académico presentado no corresponde al periodo solicitado, favor subir el documento correcto. — plazo: 15 ago. 2026'. The 'Estado del trámite' (Request Status) section shows the current stage as 'Observado' (Observed) with a deadline of 5 business days. The 'Línea de tiempo' (Timeline) section shows the request was registered on 04 ago. 2026 and the subsanation was requested on 04 ago. 2026. The 'Documentos presentados' (Submitted Documents) section shows two documents: 'Copia simple de DNI' (Not attached) and 'Récord académico actualizado' (Rejected).

Stage	Request Date	Deadline
Observado	04 ago. 2026 - 04:53 a. m.	5 días hábiles

Document	Status
Copia simple de DNI	FALTANTE
Récord académico actualizado	RECHAZADO

Figura 12: Detalle de un expediente en subsanación, con observaciones, documentos y línea de tiempo (ruta /estudiante/solicitudes/:id).

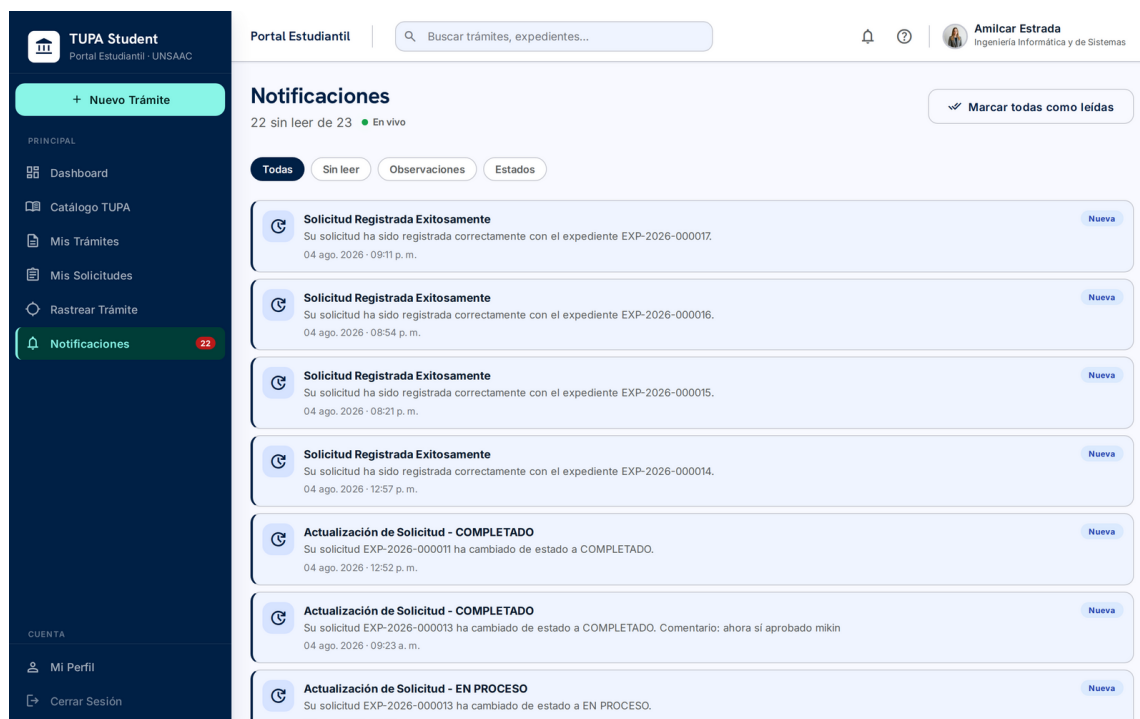


Figura 13: Centro de notificaciones del estudiante (ruta /estudiante/notificaciones).

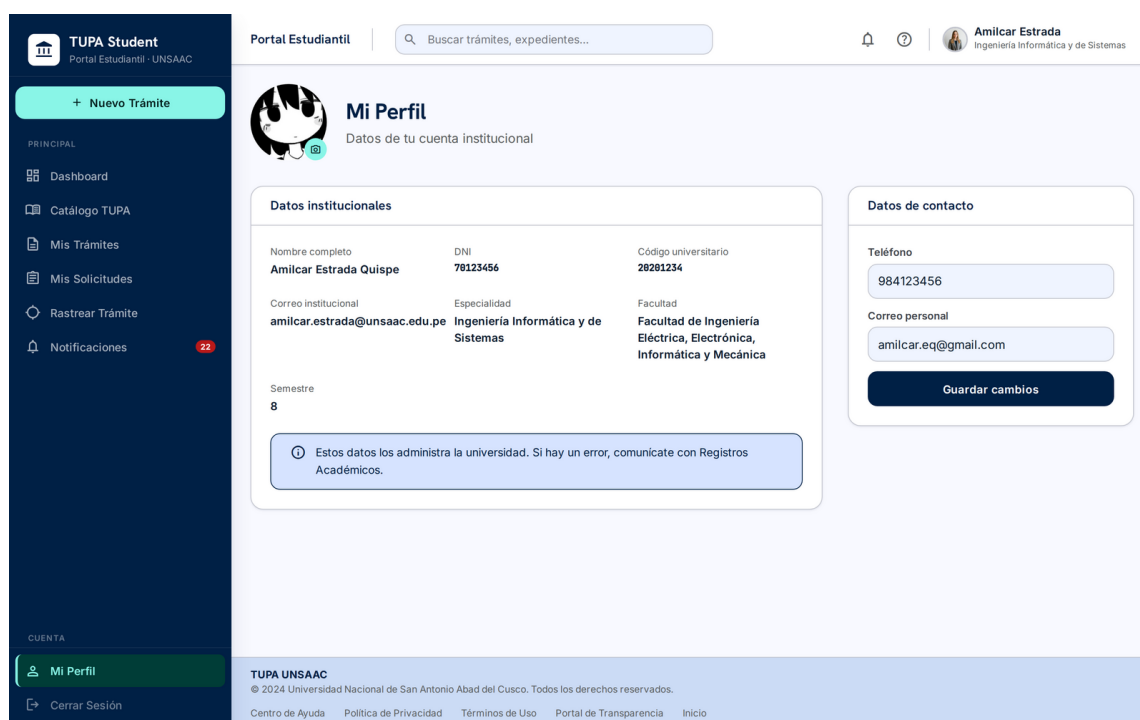


Figura 14: Perfil del estudiante; sólo los datos de contacto son editables (ruta /estudiante/perfil).

revisión final y confirmación de envío.

Figura 15: Paso 1 del asistente: selección del trámite (ruta `/tramite/paso1`).

Portal Administrativo

El portal administrativo incluye el tablero de gestión con indicadores, la cola de solicitudes pendientes, la validación documental, el detalle de expediente donde se registra la decisión, la gestión del catálogo de procedimientos, la administración de usuarios y el módulo de reportes. Estas vistas se encuentran en `AdminDashboard.jsx`, `PendingQueue.jsx`, `DocumentValidation.jsx`, `RequestDetail.jsx`, `ProcedureManagement.jsx`, `UserManagement.jsx` y `Reports.jsx` dentro de `frontend/src/pages/admin/`.

Componente Pendiente

La ruta `/seguimiento/detalle`, servida por `ProcedureTimeline.jsx`, conserva datos de ejemplo escritos directamente en el componente y no consume la API. Se documenta aquí como elemento pendiente de integración: la funcionalidad equivalente sí está resuelta y conectada en el detalle de expediente del estudiante (Figura 12) y en el resultado de la consulta pública (Figura 8).

TUPA UNSAAC — Asistente de Trámites

✕ Cancelar

123456

SelecciónRequisitosPagoDocumentosRevisarConfirmación

Revisar Requisitos

Confirma que tienes todos los documentos necesarios. Marca cada ítem para registrar que lo tienes listo.

Solicitud de Beca de Comedor

TR-005 · Gratuito · 7 días hábiles

Cambiar

☐ Ficha socioeconómica

Obligatorio

☐ Constancia de matrícula vigente

Obligatorio

← Paso anterior

Ir a Confirmación de Pago →

Progreso de preparación

Documentos confirmados 0 / 2

8 Faltan 2

Figura 16: Paso 2 del asistente: revisión de requisitos del trámite elegido.

TUPA UNSAAC — Asistente de Trámites

✕ Cancelar

123456

SelecciónRequisitosPagoDocumentosRevisarConfirmación

Confirmación de Pago

Realiza el pago de la tasa y sube tu voucher para continuar con el trámite.

Total a pagar por: Solicitud de Beca de Comedor (TR-005)

Gratuito

Referencia: TUPA-TR-005-16

Método de pago

Banco de la Nación

Cuenta corriente N.º 000-000-000000-0-00

Recomendado

Agente Bancario

Cualquier agente autorizado del Banco de la Nación

Caja de Tesorería UNSAAC

Pago presencial · Lunes a Viernes 8:00 – 16:00

Datos de depósito

Beneficiario

UNSAAC

N.º de cuenta

000-000-000000-0-00

Unidad responsable

Oficina de Bienestar Universitario

Gratuito

Monito exacto a depositar

Figura 17: Paso 3 del asistente: confirmación de pago y carga del comprobante.

Informe Final – Portal Digital TUPA UNSAAC

21

TUPA UNSAAC — Asistente de Trámites ✕ Cancelar

Selección Requisitos Pago Documentos **Revisar** Confirmación

Subir Documentos

Carga cada documento en el formato indicado. Formatos aceptados: PDF, JPG, PNG. Máximo 5 MB por archivo.

Progreso de carga 2 / 2 obligatorios

✓

Ficha socioeconómica *

Obligatorio

✓

_14ccce1c-d674-4176-a6bc-2c2dfc012b37.jpg

Eliminar

✓

Constancia de matrícula vigente *

Obligatorio

✓

79d510002e726d8bf9c9c2fde05bda0e.png

Eliminar

← Paso anterior Revisar y Enviar →

Figura 18: Paso 4 del asistente: carga de los documentos por requisito.

TUPA UNSAAC — Asistente de Trámites ✕ Cancelar

Selección Requisitos Pago Documentos **Revisar** Confirmación

Revisar y Enviar

Verifica que toda la información sea correcta antes de enviar tu solicitud.

Procedimiento seleccionado

Nombre del trámite

Solicitud de Beca de Comedor

Código TUPA

TR-885

Categoría

Bienestar Universitario

Plazo

7 días hábiles

✎ Cambiar

Datos del solicitante

Nombre completo

Amilcar Estrada Quispe

DNI

78123456

Código UNSAAC

28281234

Correo

amilcar.estrada@unsaac.edu.pe

Información de pago

✎ Editar

Resumen del envío

Trámite

Solicitud de Beca de Comedor

Documentos

2 de 2

Monto

Gratuito

Al enviar recibirás un número de expediente para rastrear el estado de tu trámite.

Figura 19: Paso 5 del asistente: revisión final antes del envío.

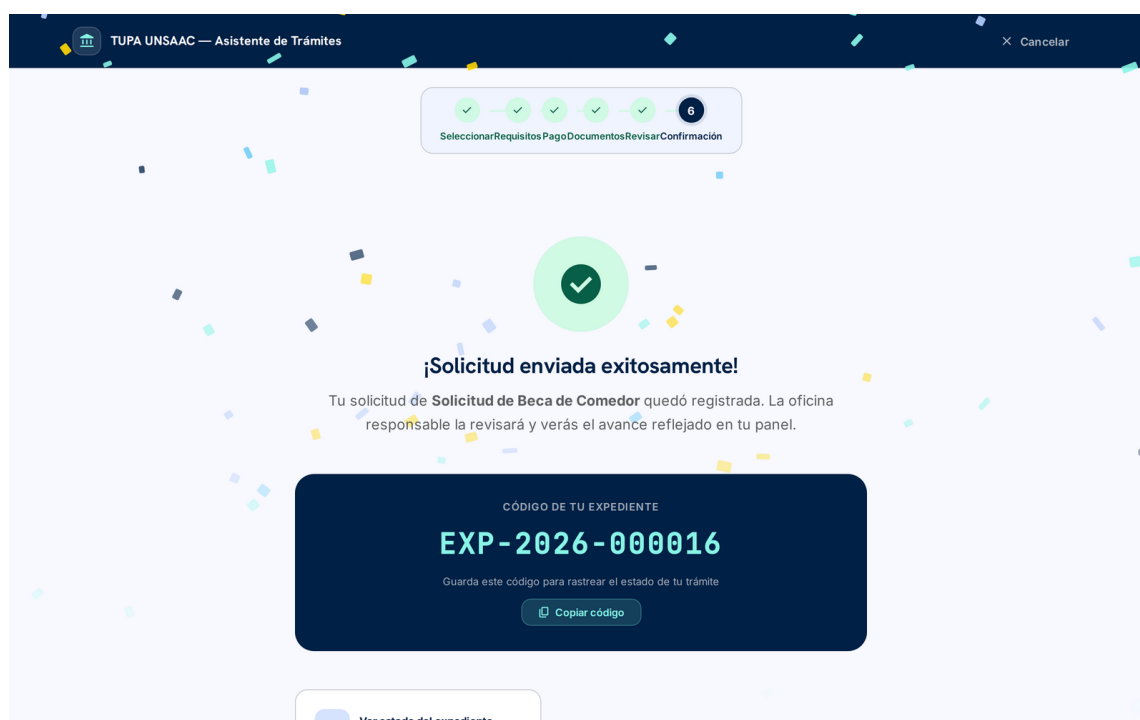


Figura 20: Paso 6 del asistente: confirmación de envío con el número de expediente generado.

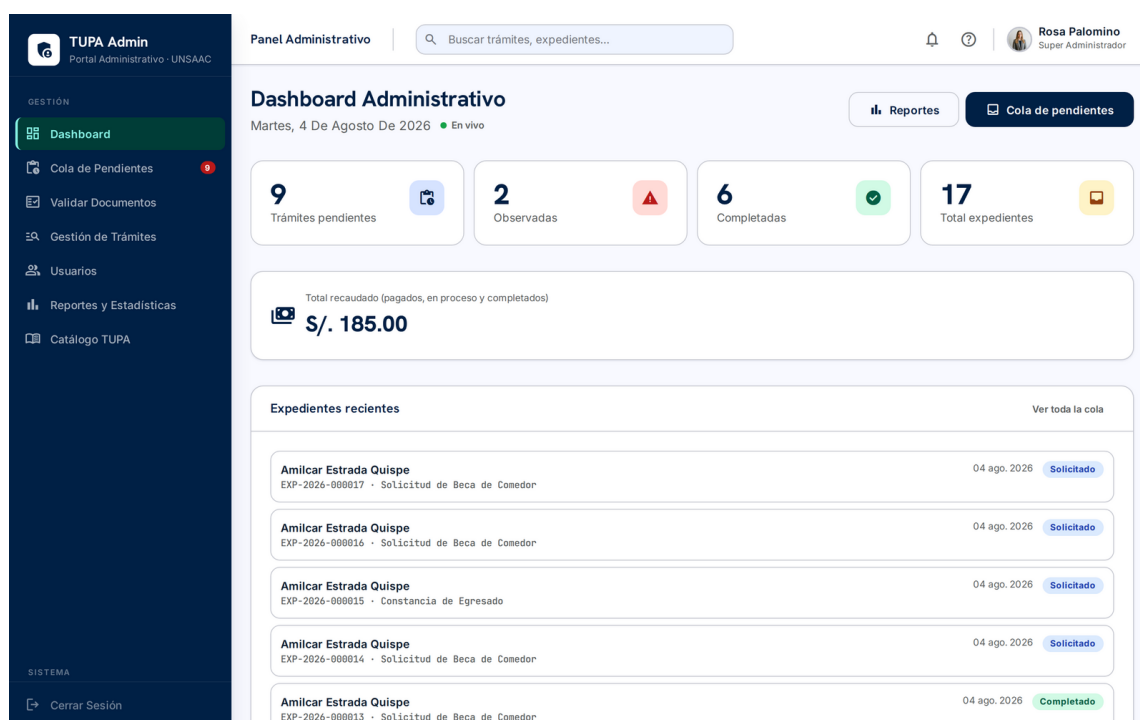


Figura 21: Tablero administrativo con indicadores agregados y expedientes recientes (ruta /admin).

TUPA Admin
Portal Administrativo - UNSAAC

Panel Administrativo | Buscar trámites, expedientes...

Cola de Pendientes
17 expedientes en esta vista • En vivo

Actualizar ahora

9 Pendientes | 2 Observadas | 17 Total expedientes | 6 Completadas

Todos | Nuevos | En proceso | Observados | Completados

Expediente, DNI o nombre...

SOLICITANTE / EXPEDIENTE	TRÁMITE	FECHA	ESTADO	ACCIÓN
Amilcar Estrada Quispe EXP-2026-000017	Solicitud de Beca de Comedor	04 ago. 2026	Solicitado	Revisar
Amilcar Estrada Quispe EXP-2026-000016	Solicitud de Beca de Comedor	04 ago. 2026	Solicitado	Revisar
Amilcar Estrada Quispe EXP-2026-000015	Constancia de Egresado	04 ago. 2026	Solicitado	Revisar
Amilcar Estrada Quispe EXP-2026-000014	Solicitud de Beca de Comedor	04 ago. 2026	Solicitado	Revisar
Amilcar Estrada Quispe EXP-2026-000013	Solicitud de Beca de Comedor	04 ago. 2026	Completado	Revisar
Amilcar Estrada Quispe EXP-2026-000012	Constancia de Matrícula	04 ago. 2026	Completado	Revisar
Amilcar Estrada Quispe EXP-2026-000011	Constancia de Matrícula	04 ago. 2026	Completado	Revisar

Figura 22: Cola de solicitudes pendientes de atención (ruta /admin/cola).

TUPA Admin
Portal Administrativo - UNSAAC

Panel Administrativo | Buscar trámites, expedientes...

Validar Documentos
Expedientes esperando revisión documentaria • En vivo

Actualizar

Solicitado | Verificando pago | Pagado | En proceso | En subsanación

Solicitud de Beca de Comedor
EXP-2026-000017
Amilcar Estrada Quispe
DNI 70123456 - 04 ago. 2026
Revisar documentos

Solicitud de Beca de Comedor
EXP-2026-000016
Amilcar Estrada Quispe
DNI 70123456 - 04 ago. 2026
Revisar documentos

Constancia de Egresado
EXP-2026-000015
Amilcar Estrada Quispe
DNI 70123456 - 04 ago. 2026
Revisar documentos

Solicitud de Beca de Comedor
EXP-2026-000014
Amilcar Estrada Quispe
DNI 70123456 - 04 ago. 2026
Revisar documentos

Constancia de Matrícula
EXP-2026-000009
Amilcar Estrada Quispe
DNI 70123456 - 04 ago. 2026
Revisar documentos

Constancia de Matrícula
EXP-2026-000008
Amilcar Estrada Quispe
DNI 70123456 - 04 ago. 2026
Revisar documentos

Solicitud de Beca de Comedor
EXP-2026-000003
Jose Mamani Tito
DNI 72345678 - 04 ago. 2026
Revisar documentos

Figura 23: Validación documentaria de los archivos cargados (ruta /admin/validacion).

TUPA Admin
Portal Administrativo - UNSAAC

Panel Administrativo | Buscar trámites, expedientes...

GESTIÓN

- Dashboard
- Cola de Pendientes **9**
- Validar Documentos
- Gestión de Trámites
- Usuarios
- Reportes y Estadísticas
- Catálogo TUPA

SISTEMA

- Cerrar Sesión

Convalidación de Cursos
EXP-2026-000002

Solicitante

Nombre	DNI	Código universitario
Maria Huamani Condori	71234567	20191122
Correo	Teléfono	Fecha de solicitud
maria.huamani@unsaac.edu.pe	984234567	04 ago. 2026 · 04:53 a. m.

Pago

Monto	N.º de recibo	Fecha de pago
S/. 60.00	REC-0002	10 jul. 2026
Comprobante	No adjuntado	

Documentos (2 de 2 requisitos) | Aprobar todos | Limpiar

Silabo del curso de origen	Obligatorio	✓	✕
silabo_curso_origen.pdf	04 ago. 2026	PENDIENTE	
Certificado de notas de la universidad de origen	Obligatorio	✓	✕
certificado_notas.pdf	04 ago. 2026	APROBADO	

Decisión

Comentario para el estudiante
Obligatorio al observar o rechazar

Para que el expediente pase a **Completado**, marca todos los documentos obligatorios como aprobados.

Aprobar

Marcar en proceso

Observar

Rechazar

Observaciones emitidas

El silabo debe estar sellado por la universidad de origen.

Figura 24: Detalle de expediente y registro de la decisión administrativa (ruta /admin/solicitudes/:id).

TUPA Admin
Portal Administrativo - UNSAAC

Panel Administrativo | Buscar trámites, expedientes...

GESTIÓN

- Dashboard
- Cola de Pendientes **9**
- Validar Documentos
- Gestión de Trámites**
- Usuarios
- Reportes y Estadísticas
- Catálogo TUPA

SISTEMA

- Cerrar Sesión

Gestión de Trámites
6 vigentes de 6 en el catálogo

Buscar trámite...

Constancia de Egresado Vigente TR-002 Documento que certifica la culminación de estudios. S/. 35.00 5 días hábiles Certificados y Constancias Desactivar del catálogo	Constancia de Matrícula Vigente TR-001 Documento que certifica la matrícula vigente del estudiante. S/. 25.00 3 días hábiles Certificados y Constancias Desactivar del catálogo	Constancia de Tercio Superior Vigente TR-900 Certifica la pertenencia al tercio superior. S/. 40.00 4 días hábiles Certificados y Constancias Desactivar del catálogo
Convalidación de Cursos Vigente TR-004 Trámite para convalidar cursos de movilidad estudiantil. S/. 60.00 10 días hábiles Movilidad Estudiantil Desactivar del catálogo	Diploma de Título Profesional Vigente TR-003 Emisión del diploma de título profesional. S/. 250.00 30 días hábiles Grados y Títulos Desactivar del catálogo	Solicitud de Beca de Comedor Vigente TR-005 Solicitud de acceso a beca de comedor universitario. Gratuito 7 días hábiles Bienestar Universitario Desactivar del catálogo

TUPA UNSAAC
© 2024 Universidad Nacional de San Antonio Abad del Cusco. Todos los derechos reservados.
Centro de Ayuda | Política de Privacidad | Términos de Uso | Portal de Transparencia | Inicio

Figura 25: Gestión del catálogo de trámites del TUPA (ruta /admin/procedimientos).

TUPA Admin
Portal Administrativo - UNSAAC

Panel Administrativo | Buscar trámites, expedientes...

GESTIÓN

- Dashboard
- Cola de Pendientes
- Validar Documentos
- Gestión de Trámites
- Usuarios**
- Reportes y Estadísticas
- Catálogo TUPA

SISTEMA

- Cerrar Sesión

Usuarios
7 usuarios registrados

Buscar por DNI, nombre o correo.

NOMBRE	DNI	CÓDIGO	CORREO INSTITUCIONAL	REGISTRO	ESTADO	ACCIÓN
Fidel Castro Pari	60434260	231865	231865@unsaac.edu.pe	04 ago. 2026	Inactivo	Activar
Rosa Testeo Demo	99887766	20260099	rosa.testeo@unsaac.edu.pe	04 ago. 2026	Activo	Desactivar
Lucia Quispe Flores	73456789	20180567	lucia.quispe@unsaac.edu.pe	04 ago. 2026	Inactivo	Activar
Maria Huamani Condori	71234567	20191122	maria.huamani@unsaac.edu.pe	04 ago. 2026	Activo	Desactivar
Amilcar Estrada Quispe	70123456	20201234	amilcar.estrada@unsaac.edu.pe	04 ago. 2026	Activo	Desactivar
Carlos Cahuana Puma	74567890	20211890	carlos.cahuana@unsaac.edu.pe	04 ago. 2026	Activo	Desactivar
Jose Mamani Tito	72345678	20221001	jose.mamani@unsaac.edu.pe	04 ago. 2026	Activo	Desactivar

TUPA UNSAAC
© 2024 Universidad Nacional de San Antonio Abad del Cusco. Todos los derechos reservados.
Centro de Ayuda | Política de Privacidad | Términos de Uso | Portal de Transparencia | Inicio

Figura 26: Administración de usuarios registrados (ruta /admin/usuarios).

TUPA Admin
Portal Administrativo - UNSAAC

Panel Administrativo | Buscar trámites, expedientes...

GESTIÓN

- Dashboard
- Cola de Pendientes
- Validar Documentos
- Gestión de Trámites
- Usuarios
- Reportes y Estadísticas**
- Catálogo TUPA

SISTEMA

- Cerrar Sesión

Reportes y Estadísticas
Calculado sobre 17 expedientes • En vivo

Actualizar

17
Total expedientes

9
Pendientes

6
Completadas

S/. 185.00
Recaudado

Expedientes por estado

Estado	Cantidad
Solicitado	7
Completado	6
Observado	1
En subsanación	1
En proceso	1
Borrador	1

Por tipo de trámite

Tipo de trámite	Cantidad	Valor
Constancia de Matricula	7	S/. 175.00
Solicitud de Beca de Comedor	5	Gratuito
Constancia de Egresado	3	S/. 105.00
Convalidación de Cursos	1	S/. 60.00
Diploma de Título Profesional	1	Gratuito

TUPA UNSAAC
© 2024 Universidad Nacional de San Antonio Abad del Cusco. Todos los derechos reservados.
Centro de Ayuda | Política de Privacidad | Términos de Uso | Portal de Transparencia | Inicio

Figura 27: Reportes y estadísticas del sistema (ruta /admin/reportes).

Seguridad

El sistema implementa los siguientes mecanismos de seguridad a nivel de API:

- **Autenticación:** el middleware `authenticateToken` intercepta las rutas protegidas, valida el token JWT recibido en la cabecera `Authorization: Bearer <token>` y adjunta la identidad del usuario a la petición.
- **Autorización por rol:** el middleware `requireRole('ADMIN')` restringe el acceso a las rutas bajo `/api/admin` a los tokens cuyo campo `role` sea `ADMIN`, devolviendo `403` en caso contrario.
- **Verificación de pertenencia:** además del rol, los servicios de solicitudes y documentos comprueban que el recurso pertenezca al usuario que lo pide, distinguiendo `404` (no existe) de `403` (existe pero es de otro usuario).
- **Cifrado de contraseñas:** las contraseñas se almacenan como `hash bcrypt` con 10 rondas de sal, nunca en texto plano. La política exige entre 8 y 64 caracteres con minúscula, mayúscula, dígito y símbolo.
- **Sesiones de doble token:** se emiten un token de acceso de 15 minutos y uno de renovación de 7 días, firmados con secretos distintos. El token de renovación se persiste en la tabla `refresh_token` y rota en cada uso: el anterior queda revocado. Cambiar la contraseña elimina todas las sesiones abiertas del usuario.
- **Limitación de intentos:** `rateLimit.middleware` restringe por IP los endpoints de inicio de sesión, registro y recuperación de contraseña, respondiendo `429` con cabecera `Retry-After` al superar el umbral. Los códigos de recuperación admiten un máximo de cinco intentos antes de invalidarse.
- **No divulgación de cuentas:** el inicio de sesión devuelve el mismo mensaje ante usuario inexistente y contraseña incorrecta, y la verificación de la cuenta activa se realiza después de comprobar la contraseña. Los endpoints de recuperación y reenvío responden siempre con un mensaje genérico.
- **Validación de archivos:** `multer` acepta únicamente PDF, JPG, JPEG y PNG, con un límite de 5 MB por archivo. Los archivos se guardan en columnas `BYTEA` de PostgreSQL y no en el sistema de archivos, dado que el backend se despliega

sobre una plataforma *serverless* de disco efímero.

- **Prevención de inyección SQL:** todas las consultas a la base de datos se ejecutan mediante sentencias parametrizadas.
- **Manejo centralizado de errores:** el middleware de errores traduce los códigos de restricción de PostgreSQL a respuestas 400/409 y sustituye por un mensaje genérico cualquier error 500, evitando filtrar detalles internos al cliente.

Limitación conocida.. La política de CORS se configura desde la variable `CLIENT_ORIGIN` como una lista de orígenes separados por comas; cuando dicha variable no está definida, el backend acepta peticiones desde cualquier origen. Esta decisión se tomó para permitir la demostración en red local desde una segunda máquina, pero en un despliegue de producción la variable debe fijarse explícitamente.

Despliegue

El proyecto contempla dos modos de ejecución. Para desarrollo y demostración, el backend se ejecuta con Node.js sobre el puerto 3000 y el frontend se sirve con Vite sobre el puerto 5173; el cliente resuelve la URL de la API tomando el mismo nombre de host desde el que se sirve la página y el puerto 3000, de modo que la demostración funciona en red local sin reconfiguración. Para el despliegue, el repositorio incluye un archivo `vercel.json` y el backend está adaptado a un entorno *serverless*: por eso ningún archivo se persiste en disco y tanto los documentos como los avatares se almacenan en PostgreSQL. La base de datos utilizada durante el desarrollo está alojada en un servicio gestionado externo. Las variables de entorno necesarias se registran en el archivo `.env` del backend e incluyen la cadena de conexión a PostgreSQL, los secretos de firma JWT, el puerto HTTP y las credenciales del servidor de correo saliente.

Pruebas

La verificación del sistema se realizó con pruebas automatizadas mediante Jest y Supertest. La suite se organiza por módulo, cubre los siete módulos del backend y consta de **6 suites** con **169 pruebas**, todas en verde en la última ejecución. Se emplean dos

estrategias complementarias:

- **Pruebas de integración HTTP:** levantan la aplicación Express con `supertest` y sustituyen la capa de servicios por dobles de prueba (`jest.mock`). Ejercitan el enrutado, los middlewares de autenticación y autorización, la validación de archivos y la forma de las respuestas sin requerir una base de datos activa, lo que las hace reproducibles en cualquier máquina y en integración continua.
- **Pruebas unitarias sin dobles:** ejecutan el código real de las utilidades, que son funciones puras o dependen únicamente de un cliente de base de datos simulable con un objeto. Con ello se verifica lógica de negocio efectiva y no sólo el cableado entre capas.

Cuadro 5: Organización de la suite de pruebas automatizadas del backend.

Archivo	Casos	Cobertura
<code>auth.test.js</code>	22	Health check, inicio de sesión (incluidos 401 y 403), registro, activación e idempotencia del enlace, renovación y cierre de sesión, recuperación de contraseña y límite de intentos.
<code>users-procedures.test.js</code>	42	Perfil y campos no editables, cambio de contraseña, avatares, catálogo y filtros, y el flujo completo de solicitudes con sus reglas de pertenencia.
<code>middleware.test.js</code>	33	Autenticación sin token, con firma inválida o expirada; guardián de rol sobre las ocho rutas administrativas; ruta inexistente; traducción de errores de PostgreSQL; validación de tipo y tamaño de archivo.
<code>utils.test.js</code>	32	Código real de <code>validate</code> , <code>generateExpediente</code> , <code>jwt.util</code> y <code>asyncHandler</code> .
<code>admin.test.js</code>	21	Estadísticas, cola de solicitudes, decisión administrativa con sus validaciones y conflictos, gestión de trámites y usuarios.
<code>documents-notifications.test.js</code>	19	Visualización y eliminación de documentos con control de pertenencia, listado y marcado de notificaciones.

Además de los casos de camino feliz, la suite verifica de forma explícita los caminos de error: peticiones sin token (401), token con firma inválida o expirado (403), rol insuficiente sobre cada una de las ocho rutas de `/api/admin` (403), identificadores mal formados (400), recursos inexistentes (404), conflictos de estado al reenviar una solicitud ya enviada o al decidir sobre un expediente cerrado (409) y exceso de intentos

en los endpoints de autenticación (429). Se comprueba también que un error inesperado responda 500 con un mensaje genérico, sin filtrar detalles internos de la base de datos. La suite se ejecuta con `npm test` desde el directorio `backend/`.

```
Test Suites: 6 passed, 6 total
Tests:      169 passed, 169 total
Snapshots:  0 total
Time:       42.164 s
Ran all test suites.
```

Figura 28: Ejecución completa de la suite de 169 pruebas automatizadas.

Historial de Commits

El desarrollo del proyecto ha transitado por diez hitos principales documentados en el control de versiones. Estos reflejan la construcción incremental de los componentes de infraestructura (almacenamiento, autenticación, integración de servicios), la verificación de funcionalidad mediante pruebas, y la preparación para despliegue.

```
$ git log --oneline -10
549c080 Memory Storage integrado
d09a8a6 Almacenamiento de Documentos
2fedc10 Integracion Sign Up
ee7de7d Merge branch main
ef4714b Informe
d0991cd checkpoint: demo
5a199a6 Sistema completo
4309ebd Test y pruebas automaticas
bedff51 Seed Actualizada
```

Figura 29: Historial de commits del repositorio, mostrando los diez últimos puntos de control.

Conclusiones

1. El uso de una arquitectura desacoplada entre el frontend en React y el backend en Express permitió desarrollar y probar ambas capas de forma independiente, comunicándose únicamente a través de contratos de API bien definidos.
2. El modelo de datos relacional en PostgreSQL, con restricciones CHECK sobre los campos de estado, permitió delegar parte de la validación del flujo del trámite a



la propia base de datos, reduciendo la posibilidad de estados inconsistentes.

Recomendaciones

1. Para una versión posterior al alcance de demo, se recomienda incorporar pruebas automatizadas (unitarias y de integración) sobre los servicios de la API, particularmente sobre la lógica de cambio de estado de una solicitud.
2. Se recomienda evaluar la integración con un proveedor de pagos real y con el sistema de autenticación institucional de la UNSAAC para una eventual puesta en producción.



Referencias Bibliográficas

- Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). Consultado el 4 de agosto de 2026, desde <https://www.rfc-editor.org/rfc/rfc7519>
- Meta Platforms, Inc. (2026). *React Documentation*. Consultado el 4 de agosto de 2026, desde <https://react.dev/>
- OpenJS Foundation. (2026). *Express - Node.js web application framework*. Consultado el 4 de agosto de 2026, desde <https://expressjs.com/>
- The PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation*. Consultado el 4 de agosto de 2026, desde <https://www.postgresql.org/docs/>